
Less is More: Early Stopping Rollout for On-Policy Distillation

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 On-policy distillation scores every token of the student’s rollout with the teacher.
2 We identify a subtlety that changes the picture: the teacher’s score at position t is
3 conditioned on the student’s own previously generated tokens, so early in a rollout
4 the teacher judges *how to approach the problem*, whereas later it can only judge *how*
5 *to continue what the student has already written*. Given a sufficiently long student
6 rollout, the teacher’s accuracy collapses toward the student baseline, indicating
7 that late-position supervision is far weaker than early-position supervision. We
8 therefore propose *Early Stopping Rollout* (ESR): restrict both the rollout and the
9 reverse-KL loss to the first N response tokens. Across tasks, model families, LoRA
10 and full fine-tuning, and a range of teacher scales, ESR matches or exceeds full-
11 sequence distillation nearly everywhere, sometimes surpasses the teacher, trains
12 far faster with far less memory, and avoids the collapse modes of full-sequence
13 training. The obvious explanation—ESR as a proxy for high-KL or high-entropy
14 tokens—fails: every scalar-saliency selector we tried underperforms ESR, and
15 most underperform plain full-sequence. Position is an independent, load-bearing
16 axis—it is where the planning tokens live, and a short reverse-KL loss there is
17 enough to improve the entire response.

18 1 Introduction

19 On-policy distillation (OPD) has emerged as a dominant paradigm for model distillation in industrial
20 practice. The student generates its own rollouts τ , which are then scored by the teacher: at each token,
21 the teacher’s probability $\pi_{teacher}(\tau_{student} \mid x_{prompt})$ serves as the soft target for the student [Agarwal
22 et al., 2024, Gu et al., 2024]. Viewed through an RL lens, this is equivalent to using the teacher as a
23 dense, token-level reward model that judges the student’s own behavior on a given prompt. The final
24 objective is then a simple *uniform average* of the per-token reverse-KL across the trajectory.

25 This uniform averaging implicitly assumes that the teacher’s signal is equally reliable at every position
26 in the trajectory. We argue that this assumption is unfounded, and that it breaks down precisely
27 because of the autoregressive structure of LLMs. At the first token, the teacher’s score is conditioned
28 only on the prompt: $\pi_{teacher}(\tau_{student}^{t=1} \mid x_{prompt})$ - indeed what we expect the teacher to score on. At
29 a later position m , however, it becomes conditioned also on the student’s own previously generated
30 tokens: $\pi_{teacher}(\tau_{student}^{t=m} \mid x_{prompt}, \tau_{student}^{t=1:m})$. This conditioning context is off-policy to the teacher
31 model and may even be out-of-distribution (OOD), and grows more so with position.

32 To probe how severe this drift is, we run a preliminary experiment that measures how the teacher
33 behaves when continuing from an early-stopped student rollout. As shown in Figure 1, the teacher’s
34 quality collapses toward the student’s within roughly 300 tokens. If the teacher’s late-position signal
35 is unreliable, the natural question is whether early tokens alone are systematically more informative
36 — and whether training on them is sufficient.

Motivated by this, we propose Early Stopping Rollout(ESR): restrict the student rollout to its first N tokens (e.g., 50 or 100) and compute the distillation loss only on this early window. The change is a single line in any on-policy distillation loop. Despite its simplicity, ESRconsistently outperforms full-rollout OPD across tasks (math, code, function calling), training regimes (LoRA, full fine-tuning), model scales (students 1.5B–7B, teachers 1.7B–14B), and model families (Qwen, Gemma), while reducing wall-clock cost by up to $24\times$ and peak training memory by up to $4\times$. We further observe that full-rollout OPD frequently destabilizes and collapses during training, whereas ESRdoes not collapse in any of our settings. Perhaps most surprisingly, although the student is normally upper-bounded by the teacher’s performance, ESR-trained students often *exceed* the teacher.

Such a small change yields such large gains probe us to investigate deeper. We have verified that later position tokens suffer from signal quality issues. But we want to further validate whether the effectiveness of earlier tokens is intrinsically caused by better KL and entropy signals, because we also find that early tokens generally have larger KL and entropy. We test selectors that pick tokens by high KL, high entropy or combined regardless of position; all underperform ESR, with most underperforming full-rollout. Therefore, although we don’t exclude KL and entropy as auxillary factors, we confirm that the position is a an independent factor rather a saliency proxy. The late position tokens have signal quality issues in themselves. Furthermore, we identified two other mechanisms that may be working together: (i) *Strategy learning*: we find content-wise the early tokens tend to encode strategies and plans, while later tokens tend to focus on specific executions. It may be better to learn just the planning part from the teacher and leave the execution to its own way. (ii) *Convergence Cascade*: after training on the early window, KL divergence on the *untrained* late tokens also drops by 30–40%; the KL divergence does not have to see late positions to repair them.

These mechanisms explain why ESRcan beat full-rollout. They do not explain why ESRsometimes even *exceeds* the teacher — a stronger phenomenon we observe on most of our main experiment Table 1. We trace this to the mode-seeking nature of reverse-KL: the student commits to a subset of supported teacher modes instead of chasing all of them. Our ESRloss helps it steer toward a non-dominant but correct mode rather than the teacher’s argmax. Empirically, we find ESR-trained students commit harder to their own choices than full-rollout students yet agree *less* with the teacher’s argmax, and produce trajectories $\sim 3\times$ shorter than full-rollout and $\sim 2\times$ shorter than the base student (§5.3). It learns the teacher’s strategy but preserves its succinct style, leading to better compound behaviors. This finding showcases a potential path of superceding the teacher model in distillation that worth further investigations.

Our contributions are summarized below:

1. **Early Stopping Rollout(ESR) outperforms full-rollout on-policy distillation.** A one-line change—restricting the rollout length to the first N response tokens—beats full-sequence distillation across tasks, model families, scales, and training regimes, while being dramatically more efficient and far more stable to train. It can even exceed the teacher’s own performance (Sections 3–4.3).
2. **A systematic investigation of why it works.** First, through a token-selection ablation, we show that the gain is not “high KL” or “high entropy” in disguise: position itself is a load-bearing axis. We also identify strategy learning, and convergence cascading to be potential factors. Furthermore, we explain why ESRoften surpasses the teacher: by exploiting the mode-seeking behavior of reverse-KL training, ESRpushes the student toward a better, non-dominant sub-mode (Section 5).

2 Why Late-Position Supervision Is Unreliable

In on-policy distillation, the teacher model scores the student’s rollout at each position t via $\pi_t(\cdot | x, y_{<t}^{\text{student}})$ and averages these scores uniformly across the rollout. The conditioning context $y_{<t}^{\text{student}}$, however, is whatever this particular student happened to write at this particular moment of training. As t grows it drifts away from any distribution the teacher itself was trained on—off-policy to the teacher and, for long enough or atypical enough rollouts, plausibly out-of-distribution.

To verify the intuition, we feed the teacher a k -token student-generated rollout on MATH-500 and letting it continue autoregressively, the teacher’s avg@4 ($n=4$, $t=0.7$) falls from its 65.30% unconditional baseline to 62.70% at $k=100$ and 51.75% at $k=300$ —essentially student-baseline

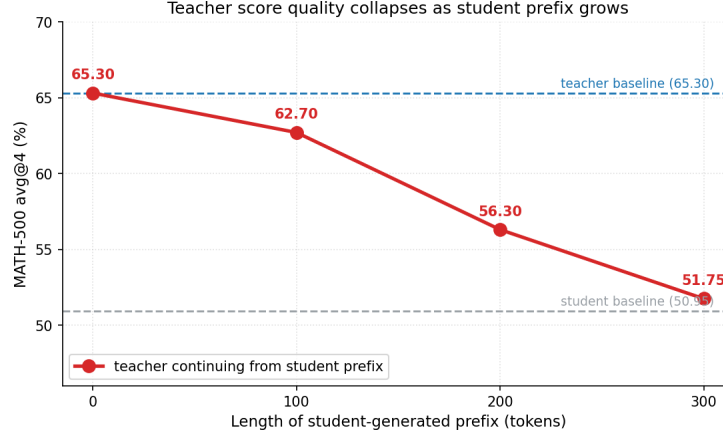


Figure 1: **The teacher loses accuracy quickly as the student-generated rollout grows in a few hundred of tokens.** MATH-500, avg@4 ($n=4$, $t=0.7$). Teacher = Qwen3-1.7B; student = Qwen2.5-Math-1.5B. After ~ 300 student tokens the teacher has effectively been dragged down to student-baseline performance.

(Figure 1). Late-position teacher scores are not independent assessments of the prompt; it is a guess at how to continue a trajectory the student has already committed to.

The problem compounds during training because the student’s policy is moving. Early on, the student’s rollout is far from the teacher; late-position targets are fitted under a strongly off-policy prefix. As training progresses and earlier tokens approach the teacher’s distribution, the right late-position behavior changes too—continuations that were sensible above an off-policy prefix are no longer sensible above an on-policy one. But the late-position habits learned earlier do not cleanly erase: gradient descent gives no guarantee of unlearning, and in LoRA’s low-rank setting it updates make redirection compete for capacity that may already be spent. Full-rollout training thus chases a moving late-position target while carrying ballast from earlier ones. This insight expectes training stability issues for full-rollout OPD – a prediction we later verify in §4.3

3 Our Method: Early Stopping Rollout (ESR) On Policy Distillation

3.1 Formulation

Let π_s denote the student and π_t the teacher. In standard on-policy reverse-KL distillation, the student generates a response $\mathbf{y} = (y_1, \dots, y_T)$ conditioned on prompt x , and the loss is

$$\mathcal{L}_{\text{full}} = \mathbb{E}_{\mathbf{y} \sim \pi_s(\cdot|x)} \left[\sum_{t=1}^T \text{KL}(\pi_s(\cdot | x, \mathbf{y}_{<t}) \parallel \pi_t(\cdot | x, \mathbf{y}_{<t})) \right]. \quad (1)$$

ESR (position cutoff N , with $N \ll T$ in practice) truncates the student rollout to its first N tokens, and the loss is computed over exactly those tokens:

$$\mathcal{L}_{\text{ESR}}(N) = \mathbb{E}_{\mathbf{y} \sim \pi_s(\cdot|x), |\mathbf{y}| \leq N} \left[\sum_{t=1}^{\min(N, |\mathbf{y}|)} \text{KL}(\pi_s(\cdot | x, \mathbf{y}_{<t}) \parallel \pi_t(\cdot | x, \mathbf{y}_{<t})) \right]. \quad (2)$$

If the student emits EOS before position N , the rollout terminates naturally. Everything else—generation temperature, LoRA target modules, optimizer, scorer—is unchanged from the standard on-policy KD loop.

Table 1: **Main results: ESR dominates Full Rollout across tasks and families.** Best score across training steps; metrics defined in §4 setup. ★: ESR surpasses the teacher reference. **Full Rollout** numbers mark the best step before degradation; ↓ flags cells where Full Rollout further decays with training (more details in Appendix C).

Pair	Method	MATH-500		Coding		BFCL
		avg@4	pass@4	HE	HE+	full_acc
Qwen2.5-Math 1.5B → Qwen3 1.7B	Student baseline	50.95	72.80	31.10	26.20	2.70
	Teacher reference	65.30	77.00	39.60	36.00	54.00
	Full Rollout	62.35↓	74.60↓	40.20↓	35.40↓	58.20★
	ESR	65.85 ★	79.80 ★	42.10 ★	36.00	61.30 ★
Gemma-2 2B → Gemma-3 4B	Student baseline	13.45	28.20	23.78	20.10	73.17
	Teacher reference	66.60	74.80	20.70	20.10	72.83
	Full Rollout	22.95	31.40	22.00↓	17.70↓	76.83★
	ESR	27.20	39.40	28.70 ★	24.40 ★	79.00 ★

4 Analysis on the Advancement of ESR

4.1 Setup

Models. We evaluate across multiple student-teacher pairs. Our primary student is Qwen2.5-Math-1.5B [Qwen Team, 2024], paired with teachers of increasing scale: Qwen3-1.7B, Qwen3-4B, and Qwen3-8B [Qwen Team, 2025]. We also tested Qwen2.5-Math-7B with Qwen3-14B to test student model size generality. To test cross-family generality, we also evaluate with gemma-2-2b-it (student) and gemma-3-4b-it (teacher).

Training. All experiments use LoRA [Hu et al., 2022] ($r=32$, $\alpha=64$, all linear layers, $\text{lr} = 5 \times 10^{-5}$) with reverse KL divergence loss. Temperature 0.7 for on-policy generation. Each training step processes a batch of 16 problems with 1 rollout per problem ($n_{\text{samples}}=1$, batch size 16). We train for 200 steps on all tasks, and saving checkpoints every 50 steps. Training data are drawn from NuminaMath, CodeUltraFeedback, and glaive-function-calling-v2. Our method uses $N=100$ unless otherwise specified.

Evaluation. MATH-500 [Hendrycks et al., 2021, Lightman et al., 2023] with $n=4$ samples at temperature 0.7 (reporting avg@4), HumanEval/HumanEval+ [Chen et al., 2021, Austin et al., 2021, Liu et al., 2023] at temperature 0.0 (reporting pass@1), BFCL [Patil et al., 2025] with 600 problems reporting full accuracy: correct function name and arguments.

4.2 Overall performance

Table 1 presents the main results across tasks and model families. Table 2 presents ablation results with full finetuning. Table 3 shows model size scaling results. We report the best performance achieved across all training steps for each configuration. But notice that full rollout distillation frequently collapse to much lower performance as it trains. We denote the sessions that collapses with a down arrows.

ESRbeats full rollout across model families and tasks, and frequently surpasses teacher performance. Across the cells of Table 1, ESR beats full rollout on every cell, and in six cells ESR surpasses the teacher outright (Qwen math avg@4, pass@4; Qwen HumanEval and HumanEval+; Qwen BFCL full_acc; Gemma coding on all four columns; Gemma BFCL full_acc). The Gemma coding results have the biggest gap: full rollout drives HumanEval pass@1 to 22.00, below the 23.78 student baseline and worse than the 20.70 teacher, while ESR recovers 31.10. Qwen-math full rollout is marked ↓ because its best number (62.35 avg@4) is a non-stable best step: under $n=16$ it collapses to 46.3% by step 100 (Appendix C.6).

ESRmatches or beats full rollout across full fine-tuning and LoRA settings. Table 2 reports FullFT on MATH-500 for both Qwen and Gemma. On Qwen **Full Rollout** 58.20 is slightly better than ours ESR 56.20 avg@4, but the gap is close. For Gemma ESR still dominates **Full Rollout**

Pair	Method	avg@4	pass@4
Qwen2.5-Math 1.5B	student / teacher	50.95 / 65.30	72.80 / 77.00
→ Qwen3 1.7B	Full Rollout	58.20	75.40
	ESR	56.20	73.80
Gemma-2 2B	student / teacher	13.45 / 66.60	28.20 / 74.80
→ Gemma-3 4B	Full Rollout	13.90	25.00
	ESR	26.65	40.40

Table 2: **FullFT on MATH-500**. Best across training steps. **Bold**: ESR beats **Full Rollout**.

Pair	Method	avg@4	pass@4
Qwen2.5-Math 1.5B	student / teacher	50.95 / 77.95	72.80 / 86.40
→ Qwen3 4B	Full Rollout	67.45 [↓]	80.60
	ESR	68.95	81.00
Qwen2.5-Math 1.5B	student / teacher	50.95 / 77.70	72.80 / 87.80
→ Qwen3 8B	Full Rollout	66.75	81.60
	ESR	67.85	82.20
Qwen2.5-Math 7B	student / teacher	53.60 / 76.15	75.00 / 83.20
→ Qwen3 14B	Full Rollout	68.85 [↓]	80.00
	ESR	68.95	81.20

Table 3: **Scaling teacher and student (LoRA) on MATH-500**. [↓]: **Full Rollout** peak before degradation. **Bold**: ESR beats **Full Rollout**.

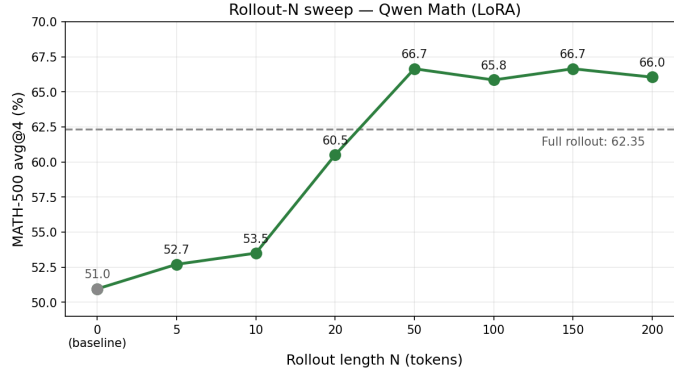


Figure 2: **Rollout Length N Sweep on MATH-500**. LoRA, Qwen2.5-Math-1.5B → Qwen3-1.7B; best avg@4 across training steps. Full rollout and the undistilled baseline are shown as horizontal references. Performance saturates for $N \in [50, 200]$ and all beat full-seq.

by +12.75pp avg@4 and +15.40pp pass@4. ESR is therefore the safer choice in both parameter regimes.

ESRscales across model sizes. The advantage persists as both teacher and student grow. Table 3 reports the results scaling both the teacher and the student. With the same student (Qwen2.5-Math-1.5B), we tried Qwen3-4B and 8B as the teacher. With Qwen3-4B, **Full Rollout** peaks at (67.45) and then *collapses* to 55.05 (Appendix C.6), while ESR stays stable and climbs to 68.95. Against the 8B teacher, ESR still edges **Full Rollout** (67.85 vs 66.75). The pattern persists at larger scale along with student model too: with a Qwen2.5-Math-7B student and Qwen3-14B teacher, **Full Rollout** again peaks at step 50 (68.85) and drifts to 62.40, while ESR holds stable across all checkpoints.

4.3 Stability of ESR

ESR is more robust than full rollout in training. Across our (pair × task × regime) matrix, full-rollout distillation degrades with training very frequently; ESR degrades nowhere. Appendix Table 14 reports each collapsing cell as a (peak → lowest) pair with the corresponding step indices. For example, Qwen-LoRA coding falls from 40.2 to 26.8; Qwen-LoRA math collapses from 65.60 at s50 to 43.80 ($\Delta = -21.80$ pp, ending below the 50.95% student baseline); Gemma-LoRA collapse from 22.0 to 11.6 ($\Delta = -10.4$ pp). Three of seven full-rollout collapses end below the student baseline. ESR generally hold a fluctuation within ±3pp of its peak across training (Tables 9, 11).

Early Stopping Rollouts is not sensitive to the choice of N . A natural question naturally occurs - how to choose where to stop? Is it sensitive? We conducted a set of sweeping experiments in Figure 2 sweeps N on MATH-500 and reveals a robust region: it reaches just as good performance starting from $N=50$ and remains stable to $N=200$. The method is not sensitive to the exact choice of N within a certain region.

Phase	Per-step wall time			Peak GPU memory		
	ESR (100 tok.)	Full rollout	Speedup	ESR (100 tok.)	Full rollout	Savings
Teacher scoring	1 s	7 s	7×	7.3 GB	14.9 GB	2.0×
Student generation	5 s	180 s	36×	7.2 GB	8.9 GB	1.2×
Student training	2 s	7 s	3.5×	9.6 GB	39.5 GB	4.1×
Total (single step)	8 s	194 s	24×	24.1 GB	63.3 GB	2.6×
Total (200 steps)	27 min	10.8 hr	24×			

Table 4: **Training efficiency.** Single A6000 (48 GB), bs=16, student teacher Qwen3-1.7B. We report the average running time and GPU memory usage across student model generation, training and teacher scoring phases. Note that the real time usage can be larger if there isn’t enough GPU to hold the student and teacher models together and requires model loading and unloading.

4.4 Efficiency of ESR

Table 4 shows that ESR achieves a 24× wall-clock speedup and reduces peak training memory by ~ 4×. The dominant cost in full-seq is autoregressive generation (180 s/step for sequences averaging ~1000 tokens); ESR generates only $N=100$ tokens (5 s/step). Note that with ESR, all the student and teacher models can be put in one A6000 GPU comfortably. In our own practice, it saves a further big time overhead of model loading and unloading that we do not report here.

5 Why Does ESR Work?

The result that a one-line change matches or exceeds full-sequence distillation — at a fraction of the compute, with stability everywhere the full-sequence baseline degrades — is surprising enough that it demands a deeper mechanism. We structure this section around three questions: (§5.1) Is the early-position effect simply a proxy for high-KL or high-entropy tokens? (§5.2) If not, what additional structural reasons make the early window privileged? (§5.3) Why does ESR sometimes *exceed* the teacher?

5.1 Is the Effectiveness of the Early Position Caused by KL and Entropy?

Figure 3 shows that early positions are simultaneously high-KL and high-entropy, so a natural alternative to positional selection is *information-theoretic* selection: pick the k tokens with the highest KL divergence or highest student/teacher entropy, regardless of position. We tested every such selector (Table 5): all underperform ESR, and most underperform full-sequence. For each strategy in Table 5 we keep the full 400-token on-policy rollout, compute the selector, and apply the reverse-KL loss only to the selected tokens.

Two findings fall out of Table 5. (i) Top-RKL=100, the selector that captures the most raw KL signal, is the *worst* (53.35%) — worse than no distillation. Increasing k to 200 only partly recovers. (ii) Every KL/entropy/combined selector underperforms full-sequence (62.35%), which in turn underperforms ESR (65.85%). These selectors all concentrate on the contaminated late region where the teacher is scoring its own continuation of the student’s previously generated tokens; the benefit of the early region is therefore *structural*, not reducible to any scalar per-token statistic.

Takeaway. ESR is not mainly caused by KL or entropy. Position is an independent, load-bearing axis — not “high-KL in disguise” and not “high-entropy in disguise”. Whatever makes the early window privileged is structural; we identify two such reasons next.

5.2 Strategy Learning and Convergence Cascade

Section 5.1 rules out scalar per-token saliency. Beyond the late-position contamination already established in §2, we identify two further structural mechanisms that jointly explain why training on the early window is so effective.

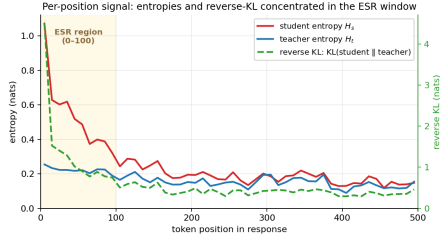


Figure 3: **Early positions carry all three — student entropy, teacher entropy, and reverse-KL — simultaneously** (Qwen, MATH-500). Natural hypothesis: ESR works because those regions carry the most signal (high KL) at the most uncertain points (high entropy). If that were the mechanism, selecting tokens by high-KL or high-entropy alone should recover the gain. Table 5 shows it does not.

Selection method	k	Best avg@4
ESR-100 (ours)	100	65.85
Full-seq (all tokens)	—	62.35
Top-RKL	100	53.35
Top-RKL	200	56.15
Top- H_t (teacher entropy)	100	63.30
Top- H_s (student entropy)	100	62.70
Top- H_s	200	53.20
RKL $\cdot H_s$	100	56.90
$H_t \cdot H_s$ (product)	100	55.35
RKL $\cdot H_t \cdot H_s$ (triple product)	100	57.90

Table 5: **Token selection strategies on MATH-500.** Qwen2.5-Math-1.5B $\rightarrow$ Qwen3-1.7B, LoRA, $n_{\text{samples}}=1$, best avg@4 across training steps. All selectors operate on full-length rollouts; ESR is the only one that additionally truncates the rollout. Baseline (no distillation) 50.95%; full-seq 62.35%. H_s : student entropy; H_t : teacher entropy; RKL: reverse KL.

199 **Strategy learning — the early window is where planning tokens live.** The early-position KL
200 spike concentrates on planning / structural tokens: the highest-KL tokens in our corpus are “To” (KL
201 6.90, $n=8806$), “First” (9.98), “Let”, “The”, “Therefore” (7.95), “Thus”, along with format-opening
202 tokens $\backslash$ [, $\backslash$ (, and code-fence ‘ ‘ ‘ (see Table 17). These tokens *select* the reasoning trajectory —
203 different openings commit the student to algebra-with-variables, factoring, or numeric-estimation
204 paths on the same problem. The teacher’s conditional distribution at these tokens is genuinely
205 multi-modal (entropy 1.07), so reverse-KL has room to move the student to a single teacher-supported
206 mode. After the plan is committed, the remainder is execution: the teacher is near-deterministic
207 (entropy 0.14) and there is little signal to extract. *Teach the plan; the execution follows* — and no
208 scalar saliency can recover the early-window region because what matters is where in the rollout a
209 token lives, not how surprising it is locally.

210 **Convergence cascade — training the early window lowers KL everywhere.** A natural worry
211 is that ESR helps only inside the trained window. The opposite holds. After ESR-200 training,
212 per-position KL between distilled student and teacher drops by 50–60% inside $[0, N]$ and by 30–40%
213 outside it (Figure 4). Comparing distilled vs. base responses on the same prompts, late tokens —
214 which receive zero direct supervision — change as much as early tokens in surface content (Jaccard to
215 baseline = 0.251 for both halves). Autoregressive generation explains the cascade: once the student
216 has committed to the right plan in the first N tokens, the remaining rollout is a lower-KL, more
217 on-policy completion for the teacher, so no gradient is needed there. The loss does not have to *see*
218 late positions to align them.

219 5.3 Why Does ESR Sometimes Exceed the Teacher?

220 ESR exceeds the teacher on several cells (Qwen math; Qwen and Gemma function calling; Gemma
221 coding, where the teacher is in fact weaker than the student). We offer a hypothesis. Reverse KL
222 $\text{KL}(\pi_s \parallel \pi_t)$ is mode-seeking: it penalizes the student for putting mass on tokens the teacher does not
223 support, but not for concentrating mass on a single supported token. When the teacher is multi-modal
224 at the planning positions—one mode being a verbose, wasteful plan and another a concise correct
225 plan—a short early-window loss can force the student to commit to the smaller, correct mode rather
226 than averaging (Figure 5). On function-calling this effect is pronounced: ESR-100 lifts Qwen BFCL
227 full_acc from 54.0% (teacher) to 61.3%, and the chained Qwen3-4B $\rightarrow$ 8B configuration raises it to
228 76.8% against a 73.0% teacher. Full-sequence training cannot achieve this because its late-position
229 loss pulls the student toward the average teacher behaviour across all supported modes, undoing the
230 concentration.

231 **Behavioral signature.** The hypothesis predicts that ESR-trained students should (a) produce
232 shorter, less-hedging rollouts than the teacher, and (b) commit hard, but on *non-argmax* teacher

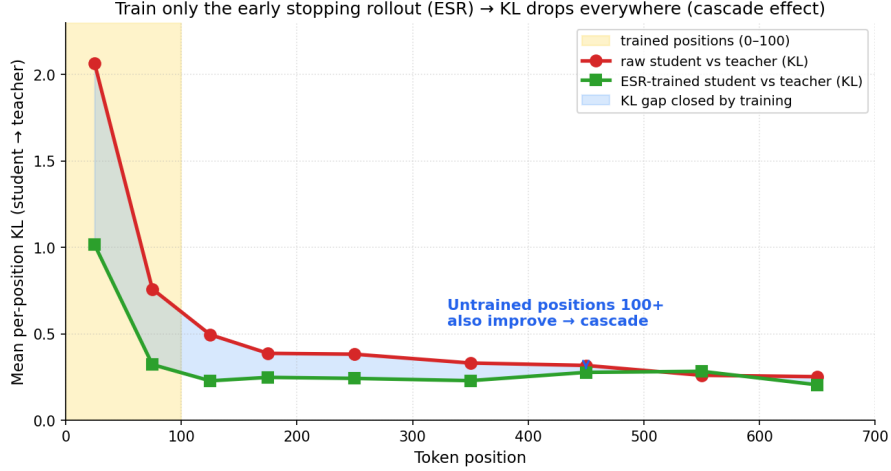


Figure 4: **The convergence cascade.** Per-position KL between distilled student and teacher, before vs. after ESR-200 training. Yellow band: positions $[0, 200]$ that actually receive training loss. Blue band: the KL gap closed by training. Positions $200+$ — which see no direct training signal — drop to the same KL as the trained region, confirming that alignment on the early window cascades through the autoregressive rollout.

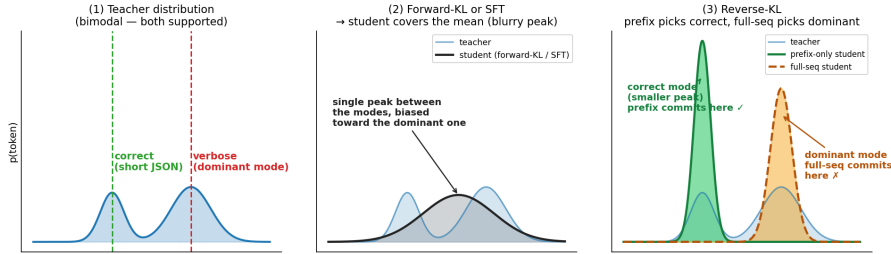


Figure 5: **Mode-seeking on the planning region.** Schematic of reverse-KL behaviour at a multi-modal planning position. The teacher supports two modes (e.g. a verbose plan and a concise correct plan); reverse KL penalises student mass outside the support but not concentration within it. A short early-window loss can collapse the student onto the better supported mode, allowing the student to exceed the teacher’s average behaviour. Full-sequence training reverts the student toward the averaged teacher across late positions, undoing the concentration.

233 modes. Table 6 confirms both. ESR-100 rollouts are shorter than the base student, the teacher, and
 234 full-seq across the entire length distribution — even the top-decile (~ 800 tokens) is below the base
 235 student’s median (~ 860). On the top-10% highest-KL positions ($n=110,894$), ESR raises student
 236 top-1 probability to 0.79 (vs 0.77 full-seq, 0.71 base) while *lowering* agreement with the teacher’s
 237 argmax to 41.9% (vs 45.7% full-seq) and raising the rate at which the student picks tokens in the
 238 teacher’s top-2 to top-5 to 47.4% (vs 44.6%). ESR commits harder, but on different teacher-supported
 239 tokens; full-seq commits toward the teacher’s argmax and inherits the teacher’s mistakes on those
 240 positions.

241 This is the mode-seeking mechanism in numbers. When one of those teacher-supported alternatives
 242 happens to be the correct mode, ESR exceeds the teacher; full-seq cannot, because supervising the
 243 late tail pulls the student toward the teacher’s argmax and the teacher’s mistakes along with it. A full
 244 proof of this hypothesis is out of scope; we note only that it is the natural reverse-KL-mode-seeking
 245 story applied to the planning region, and that it is consistent with every surpass-teacher cell we
 246 observe.

Model	Response length (tokens)					Student commitment & mode alignment			
	10th pctl	50th pctl	90th pctl	Mean	Ratio	Top-1 prob	= tch. top-1	∈ tch. top 2–5	∉ tch. top-5
Base student	~400	~860	~1,500	~990	2.2×	0.71	28.6%	59.8%	11.5%
Teacher (Q3-1.7B)	~700	~1,150	~1,800	~1,210	2.6×	—	—	—	—
Full-seq	~1,190	~1,530	~1,770	~1,480	3.2×	0.77	45.7%	44.6%	9.7%
ESR-100 (ours)	~190	~380	~800	~460	1.0×	0.79	41.9%	47.4%	10.7%

Table 6: **Behavioral signature of ESR.** *Left:* Response length distribution on MATH-500 ($n=2,000$ rollouts per model; tokens estimated as chars/3.5). ESR-100 is shorter than base, teacher, and full-seq across the whole distribution: even its long tail (90th pctl ~ 800 tok) is shorter than the base student’s typical response (50th pctl ~ 860 tok). **Ratio** = mean length relative to ESR. Full-seq mean is below its median because length is clipped at `max_new_tokens`. *Right:* Top-10% of token positions ranked by $KL[\pi_s \parallel \pi_t]$ on 100 base-student trajectories ($n=110,894$). “= tch. top-1”, “∈ tch. top 2–5”, and “∉ tch. top-5” are mutually exclusive and sum to 100% (teacher omitted because the comparison is student-vs-teacher). ESR-100 commits hardest (top-1 prob 0.79) but agrees *less* with the teacher’s argmax than full-seq, and picks teacher-supported alternatives more often.

247 6 Related Work

248 **Knowledge Distillation for Language Models.** Knowledge distillation [Hinton et al., 2015]
 249 transfers knowledge from a teacher to a smaller student via soft targets. Kim and Rush [2016]
 250 extended this to sequence models with word-level and sequence-level objectives. For autoregressive
 251 LLMs, the choice of divergence and data distribution is decisive. Gu et al. [2024] argued for
 252 reverse KL in generative LLM distillation, on the grounds that reverse KL avoids the student placing
 253 probability on low-support teacher regions; Agarwal et al. [2024] introduced Generalized Knowledge
 254 Distillation (GKD), which generates rollouts from the student (“on-policy”) and supervises with
 255 teacher logits—substantially outperforming supervised off-policy distillation on reasoning tasks. Ko
 256 et al. [2024] proposed skew KL and an adaptive off-policy schedule; Wen et al. [2023] generalized
 257 sequence-level KD to arbitrary f -divergences; Wu et al. [2025] revisited the mode- vs. mean-seeking
 258 interpretation of reverse and forward KL in the LLM setting and showed that both converge under
 259 sufficient training but differ in which distribution regions they first concentrate on. Xu et al. [2025]
 260 introduced Speculative Knowledge Distillation, which interleaves teacher-student sampling in the
 261 style of speculative decoding and filters tokens the teacher would not have produced. Our work builds
 262 directly on the on-policy reverse-KL paradigm of Gu et al. [2024] and Agarwal et al. [2024], but
 263 asks a different question: holding the divergence and data distribution fixed, *which token positions*
 264 *along the student’s rollout carry useful signal?* Our answer—only the early window (Early Stopping
 265 Rollout)—is orthogonal to the divergence-design axis these works explore.

266 **Token-Level Importance in Distillation and Reasoning.** A growing body of work demonstrates
 267 that not all tokens contribute equally to learning. Wang et al. [2025] showed that only $\sim 20\%$ of chain-
 268 of-thought tokens have high entropy, and these “forking tokens” steer reasoning paths—applying
 269 RL gradients to only these tokens matches full-gradient performance. In the RL fine-tuning setting,
 270 Vassoyan et al. [2025] demonstrated that uniform KL penalties across all token positions prevent
 271 exploration on “critical tokens” where the pretrained model is most uncertain, and proposed entropy-
 272 weighted KL relaxation that allows the model to deviate from its prior precisely where it needs to
 273 learn. Li et al. [2025] discovered a “preplan-and-anchor” mechanism in LLM attention patterns,
 274 where models first perform long-range contextual reference before producing semantic anchor tokens,
 275 enabling targeted credit assignment. Singh and Hakkani-Tür [2026] proposed greedy pruning of
 276 reasoning chains and showed that LLMs internally encode token-level functional importance for
 277 answer generation.

278 In the distillation setting specifically, several concurrent works have explored token selection. Huang
 279 et al. [2025] proposed SelecTKD, a propose-and-verify framework where the teacher verifies student-
 280 proposed tokens, applying full loss to accepted tokens and masking rejected ones. Xie et al. [2026]
 281 introduced AdaKD, which adaptively adjusts token-level temperature based on learning stability.
 282 Tavor et al. [2026] systematically disentangled selective KD along position, class, and sample axes,

proposing student-entropy-guided position selection (SE-KD) that reduces wall time by 70% without sacrificing accuracy. Kim and Baek [2026] combined entropy-based token selection with preference ranking in a dual distillation framework.

These works collectively establish that not all tokens deserve equal gradient weight, and most of them pick the “important” tokens via a scalar per-token criterion—entropy, KL, difficulty, or a verification score. Our Section 5.1 ablation directly contradicts the premise that such scalars are sufficient: selecting the same number of tokens by top-KL, top-entropy (student, teacher, AND, OR, triple-product), soft-weighted joint entropy, or top- K -renorm RKL (a vocabulary-level renormalization of RKL proposed for stabilizing distillation; exact citation to be added in camera-ready)¹ all underperform ESR and most underperform plain full-sequence training. Position is the load-bearing axis; scalar token saliencies are not. This also resolves a tension with Wang et al. [2025], who find that RL gradients concentrated on high-entropy “forking tokens” suffice: in on-policy *distillation*, the high-entropy tokens are disproportionately in the uncontaminated early window, so their result is consistent with ours once this early-window concentration of entropy is accounted for.

Training Stability and Sequence-Level Issues. Full-sequence on-policy distillation is known to suffer from instability. Holtzman et al. [2020] identified that likelihood-based training can produce bland, repetitive text—a form of neural text degeneration. Shumailov et al. [2024] demonstrated that iteratively training generative models on their own outputs leads to model collapse, where distribution tails disappear. In on-policy KD specifically, Jang et al. [2026] proposed Veto, a geometric bridge in logit space that suppresses harmful gradients on low-confidence tokens, directly addressing the instability we also observe. Our approach is complementary: rather than modifying the target distribution, we restrict the loss scope to early positions where teacher-student agreement is naturally highest, achieving stability through a simpler mechanism.

Several works have studied how sequence length affects distillation quality. Chen et al. [2025] showed that training on only the first 50% of tokens in supervised reasoning distillation retains ~91% of full-sequence performance, establishing a computation-quality tradeoff. Yan et al. [2026] addressed exposure bias and capacity misalignment in long chain-of-thought distillation through temperature-scheduled learning. Our work extends the truncation insight to the on-policy setting: we demonstrate that positional loss masking on student-generated sequences achieves equal or better performance than full-sequence loss, while providing the additional benefit of eliminating degenerate repetition patterns that are unique to on-policy training.

Curriculum Learning and Sequence Scheduling. Curriculum learning [Bengio et al., 2009] improves training by gradually increasing task difficulty. Bengio et al. [2015] applied this principle to sequence models through scheduled sampling, transitioning from teacher-forced to free-running generation during training. In the LLM pretraining setting, Li et al. [2022] showed that long sequences at training start cause extreme gradient variance, and sequence length warmup stabilizes training while enabling larger batch sizes. Pouransari et al. [2024] extended this with variable sequence length curriculum for LLM pretraining, achieving up to $3\times$ faster convergence. Our progressive position schedule—linearly increasing the position limit from 1 to 200 over training—applies the curriculum principle to the distillation loss scope rather than to the input sequence length, and represents a novel form of curriculum specific to knowledge distillation.

Reasoning Distillation. Chain-of-thought prompting [Wei et al., 2022] made intermediate reasoning the unit of transfer for distillation, and the DeepSeek-R1 release [DeepSeek-AI, 2025] established that SFT-distilling a strong reasoner into a smaller model is often competitive with direct RL on the small model. Our experiments include three reasoning tasks (MATH-500, HumanEval/MBPP, BFCL) and a 1.5B-math-student $\rightarrow$ 8B-teacher sweep; ESR surpasses the teacher on several of these cells (Qwen math, function-calling), indicating that reasoning-distillation gains beyond the teacher are possible when the mode-seeking reverse-KL loss is restricted to the planning region. This is a complementary perspective to the DeepSeek-R1 recipe, which uses fully-sequenced SFT: our analysis explains why the on-policy, early-stopped-rollout regime is both more efficient and more stable for reasoning-style chain-of-thought distillation, and why full-sequence on-policy distillation—which in principle should be the strongest setting—frequently degrades across training on coding and math.

¹TODO: resolve and cite the top- K -renorm RKL method (Fu et al., 2025).

7 Conclusion

In on-policy knowledge distillation, the teacher’s per-token supervision quality is a strong, monotonic function of position: early tokens carry strong, independent signal about reasoning strategy; late tokens, scored on the student’s own contaminated previous output, contribute weak, dependent signal that is actively harmful when trained on. A one-line intervention — restrict the loss and the rollout to the first N response tokens — yields four simultaneous wins: (i) **better**, matching or exceeding full-sequence distillation on every task/family/regime we test and surpassing the teacher in nine cells; (ii) **faster**, $20\text{--}35\times$ wall-clock with $4\times$ less GPU memory; (iii) **stabler**, eliminating HumanEval collapse, BFCL parse-rate collapse, and the MATH-500 boxed-repetition failure mode; and (iv) **structural**, in that selecting the same number of tokens by any KL or entropy saliency fails — position is an independent axis, not a proxy for scalar surprise. Teaching the student how to begin is sufficient to improve the entire response.

Limitations. Our experiments are conducted on small open-source models with a limited data budget; whether the ESR story holds at industrial scale (much larger students, teachers, and training corpora) remains unclear. We also do not test multi-modality or long-horizon tasks, both of which may exhibit different positional signal-quality patterns.

References

- Rishabh Agarwal, Nino Vieillard, Yongchao Zhou, Piotr Stanczyk, Sabela Ramos, Matthieu Geist, and Olivier Bachem. On-policy distillation of language models: Learning from self-generated mistakes. In *International Conference on Learning Representations*, 2024.
- Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.
- Samy Bengio, Oriol Vinyals, Navdeep Jaitly, and Noam Shazeer. Scheduled sampling for sequence prediction with recurrent neural networks. In *Advances in Neural Information Processing Systems*, 2015.
- Yoshua Bengio, Jérôme Louradour, Ronan Collobert, and Jason Weston. Curriculum learning. In *Proceedings of the 26th Annual International Conference on Machine Learning*, pages 41–48, 2009.
- Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- Wei-Rui Chen, Vignesh Kothapalli, Ata Fatahibaarzi, Hejian Sang, Shao Tang, Qingquan Song, Zhipeng Wang, and Muhammad Abdul-Mageed. Distilling the essence: Efficient reasoning distillation via sequence truncation. *arXiv preprint arXiv:2512.21002*, 2025.
- DeepSeek-AI. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- Yuxian Gu, Li Dong, Furu Wei, and Minlie Huang. MiniLLM: Knowledge distillation of large language models. In *International Conference on Learning Representations*, 2024.
- Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. In *Advances in Neural Information Processing Systems*, 2021.
- Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. In *NIPS Deep Learning and Representation Learning Workshop*, 2015.
- Ari Holtzman, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi. The curious case of neural text degeneration. In *International Conference on Learning Representations*, 2020.

381 Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang,
382 and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International*
383 *Conference on Learning Representations*, 2022.

384 Haiduo Huang, Jiangcheng Song, Yadong Zhang, and Pengju Ren. SelecTKD: Selective token-
385 weighted knowledge distillation for LLMs. *arXiv preprint arXiv:2510.24021*, 2025.

386 Ijun Jang, Jewon Yeom, Juan Yeo, Hyunggu Lim, and Taesup Kim. Stable on-policy distillation
387 through adaptive target reformulation. *arXiv preprint arXiv:2601.07155*, 2026.

388 Minsang Kim and Seung Jun Baek. Explain in your own words: Improving reasoning via token-
389 selective dual knowledge distillation. *arXiv preprint arXiv:2603.13260*, 2026.

390 Yoon Kim and Alexander M. Rush. Sequence-level knowledge distillation. In *Proceedings of the*
391 *2016 Conference on Empirical Methods in Natural Language Processing*, pages 1317–1327, 2016.

392 Jongwoo Ko, Sungnyun Kim, Tianyi Chen, and Se-Young Yun. DistiLLM: Towards streamlined
393 distillation for large language models. In *International Conference on Machine Learning*, 2024.

394 Conglong Li, Minjia Zhang, and Yuxiong He. The stability-efficiency dilemma: Investigating
395 sequence length warmup for training GPT models. In *Advances in Neural Information Processing*
396 *Systems*, 2022.

397 Yang Li, Zhichen Dong, Yuhan Sun, Weixun Wang, Shaopan Xiong, Yijia Luo, Jiashun Liu, Han
398 Lu, Jiamang Wang, Wenbo Su, Bo Zheng, and Junchi Yan. Attention illuminates LLM reason-
399 ing: The preplan-and-anchor rhythm enables fine-grained policy optimization. *arXiv preprint*
400 *arXiv:2510.13554*, 2025.

401 Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan
402 Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s verify step by step. *arXiv preprint*
403 *arXiv:2305.20050*, 2023.

404 Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is your code generated by
405 ChatGPT really correct? rigorous evaluation of large language models for code generation. In
406 *Advances in Neural Information Processing Systems*, 2023.

407 Shishir G. Patil, Huanzhi Mao, Fanjia Yan, Charlie Ji, Vishnu Suresh, Ion Stoica, and Joseph E.
408 Gonzalez. The Berkeley function calling leaderboard (BFCL): From tool use to agentic evaluation
409 of large language models. In *Proceedings of the 42nd International Conference on Machine*
410 *Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 48371–48392, 2025.

411 Hadi Pouransari, Chun-Liang Li, Jen-Hao Rick Chang, Pavan Kumar Anasosalu Vasu, Cem Koc,
412 Vaishaal Shankar, and Oncel Tuzel. Dataset decomposition: Faster LLM training with variable
413 sequence length curriculum. In *Advances in Neural Information Processing Systems*, 2024.

414 Qwen Team. Qwen2.5-math technical report: Toward mathematical expert model via self-
415 improvement. *arXiv preprint arXiv:2409.12122*, 2024.

416 Qwen Team. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

417 Ilia Shumailov, Zakhar Shumaylov, Yiren Zhao, Nicolas Papernot, Ross Anderson, and Yarin Gal. AI
418 models collapse when trained on recursively generated data. *Nature*, 631(8022):755–759, 2024.

419 Janvijay Singh and Dilek Hakkani-Tür. Do LLMs encode functional importance of reasoning tokens?
420 *arXiv preprint arXiv:2601.03066*, 2026.

421 Almog Tavor, Itay Ebenspanger, Neil Cnaan, and Mor Geva. Rethinking selective knowledge
422 distillation. *arXiv preprint arXiv:2602.01395*, 2026.

423 Jean Vassoyan, Nathanael Beau, and Roman Plaud. Ignore the KL penalty! boosting exploration
424 on critical tokens to enhance RL fine-tuning. In *Findings of the Association for Computational*
425 *Linguistics: NAACL 2025*, 2025.

- 426 Shenzhi Wang, Le Yu, Chang Gao, Chujie Zheng, Shixuan Liu, Rui Lu, Kai Dang, Xionghui Chen,
427 Jianxin Yang, Zhenru Zhang, Yuqiong Liu, An Yang, Andrew Zhao, Yang Yue, Shiji Song, Bowen
428 Yu, Gao Huang, and Junyang Lin. Beyond the 80/20 rule: High-entropy minority tokens drive
429 effective reinforcement learning for LLM reasoning. In *Advances in Neural Information Processing
430 Systems*, 2025.
- 431 Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc
432 Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In
433 *Advances in Neural Information Processing Systems*, 2022.
- 434 Yuqiao Wen, Zichao Li, Wenyu Du, and Lili Mou. f-divergence minimization for sequence-level
435 knowledge distillation. In *Proceedings of the 61st Annual Meeting of the Association for Computa-
436 tional Linguistics (Volume 1: Long Papers)*, pages 10817–10834, 2023.
- 437 Taiqiang Wu, Chaofan Tao, Jiahao Wang, Runming Yang, Zhe Zhao, and Ngai Wong. Rethinking
438 Kullback-Leibler divergence in knowledge distillation for large language models. In *Proceedings
439 of the International Conference on Computational Linguistics (COLING)*, 2025.
- 440 Xurong Xie, Zhucun Xue, Jiafu Wu, Jian Li, Yabiao Wang, Xiaobin Hu, Yong Liu, and Jiangn-
441 ing Zhang. LLM-oriented token-adaptive knowledge distillation. In *Proceedings of the AAAI
442 Conference on Artificial Intelligence*, 2026.
- 443 Wenda Xu, Rujun Han, Zifeng Wang, Long T. Le, Dhruv Madeka, Lei Li, William Yang Wang,
444 Rishabh Agarwal, Chen-Yu Lee, and Tomas Pfister. Speculative knowledge distillation: Bridging
445 the teacher-student gap through interleaved sampling. In *International Conference on Learning
446 Representations*, 2025.
- 447 Shaotian Yan, Kaiyuan Liu, Chen Shen, Bing Wang, Sinan Fan, Jun Zhang, Yue Wu, Zheng Wang,
448 and Jieping Ye. Distribution-aligned sequence distillation for superior long-CoT reasoning. *arXiv
449 preprint arXiv:2601.09088*, 2026.

450 A Training Efficiency

Table 7: **Training efficiency, detailed breakdown.** Per-step wall-clock time on a single A6000 (48GB). Student: Qwen2.5-Math-1.5B, LoRA, bs=16.

Teacher	Method	Gen (s)	Score (s)	Train (s)	Total (s)
Q3-1.7B	Full-seq	100–731	3–10	5–12	~280
	Ours ($N=100$)	~5	~1	~2	~8
Q3-4B	Full-seq	97–733	3–9	6–10	~210
	Ours ($N=100$)	~5	~1	~2	~8
Q3-8B	Full-seq	96–230	7–11	1–6	~170 [†]
	Ours ($N=100$)	~5	~1	~2	~8

[†] Frequent OOMs; 48GB insufficient for 8B teacher + vLLM + student.

451 B Additional Student-Teacher Pairs

Table 8: **Additional student-teacher pairs.** All use ESR-100, LoRA, 3,200 problems.

Student	Teacher	MATH-500 avg@4	BFCL full_acc	HumanEval pass@1
Qwen3-1.7B	— (baseline)	69.20%	59.80%	39.63%
Qwen3-1.7B	Qwen3-4B	69.20%	73.70%	44.51%
Qwen3-1.7B	Qwen3-8B	69.15%	74.00%	40.24%
Qwen3-4B	— (baseline)	77.95%	73.00%	73.17%
Qwen3-4B	Qwen3-8B	77.05%	76.80%	71.34%

452 C Full Experimental Results

453 C.1 Math Results: Per-Step Performance

454 Table 9 presents per-step results for the primary math experiments (LoRA, $n=1$, 3,200 problems).

Table 9: **MATH-500 per-step results.** LoRA, $n=1$, bs=16, 3,200 problems. Baseline: 50.95% avg@4.

Method	Metric	Step 50	Step 100	Step 150	Step 200
ESR-50	avg@4	62.35	66.05	66.65	64.85
	maj@4	69.40	72.00	71.00	71.20
	pass@4	77.20	79.40	81.00	79.60
ESR-100	avg@4	63.75	64.45	65.15	65.85
	maj@4	70.00	68.40	69.60	70.80
	pass@4	79.80	78.40	80.20	79.80
ESR-150	avg@4	65.35	66.65	65.30	65.75
	maj@4	66.80	67.00	66.30	67.30
	pass@4	79.00	81.00	78.20	80.00
ESR-200	avg@4	66.05	64.65	65.10	65.55
	maj@4	71.20	68.40	70.00	71.20
	pass@4	81.00	79.80	80.60	80.60

455 C.2 Math Results: Earlier Series ($n=16$, 200 problems)

Table 10: **Complete MATH-500 results, LoRA, $n=16$, bs=16.** Best per configuration in **bold**.

Config	Step 50	Step 100	Step 150	Step 200
<i>avg@4</i>				
ESR-5	55.15	55.65	56.50	56.30
ESR-10	59.50	56.40	56.30	56.00
ESR-20	60.20	57.35	56.40	56.55
ESR-50	62.45	62.15	60.50	61.90
ESR-100	61.15	63.55	64.25	63.85
ESR-150	63.55	65.70	64.80	64.40
ESR-200	63.25	66.75	64.55	65.25
Progressive	61.10	62.05	61.95	62.30
Full-seq [†]	65.60	46.30	49.70	43.80
Full-seq (first-boxed)	65.55	64.95	65.55	64.95
<i>pass@4</i>				
ESR-5	74.0	74.8	76.0	73.4
ESR-10	75.8	73.2	75.4	74.2
ESR-20	77.0	76.0	73.4	75.8
ESR-50	77.0	78.4	77.6	78.6
ESR-100	77.6	79.2	80.2	80.2
ESR-150	79.6	77.4	80.0	79.4
ESR-200	79.6	80.2	79.4	79.2
Progressive	77.2	77.8	78.2	78.8
Full-seq	80.0	72.2	77.0	74.2

[†] Last-boxed extraction; degrades due to repetition.

Table 11: **Complete coding results, LoRA.** HumanEval (HE) and MBPP pass@1.

Config	Metric	s50	s100	s150	s200	s250	s300	s350	s400
ESR-50	HE	37.8	39.0	39.6	41.5	40.2	40.9	42.1	40.9
	MBPP	52.1	48.9	46.0	46.0	47.6	46.3	46.6	46.3
ESR-100	HE	37.2	39.0	42.1	37.8	39.0	37.8	37.8	38.4
	MBPP	52.1	49.7	49.2	49.2	48.4	49.2	47.9	48.9
ESR-150	HE	36.6	35.4	36.6	39.0	41.5	39.6	38.4	37.2
	MBPP	51.3	50.0	48.4	50.5	49.5	50.3	49.7	48.7
Full-seq	HE	40.2	31.7	32.3	32.9	27.4	28.0	26.8	26.8
	MBPP	52.6	48.9	49.5	48.1	47.9	47.1	48.4	47.6

Table 12: **Function calling results (BFCL), LoRA.** Name accuracy / Full accuracy / Parse rate. Best full_acc in **bold**.

Method	Best Step	Name Acc	Full Acc	Parse Rate
Baseline	—	9.70%	2.70%	24.20%
Teacher (Qwen3-1.7B)	—	75.30%	54.00%	75.30%
ESR-50	200	95.20%	57.20%	98.30%
ESR-100	100	86.20%	61.30%	91.30%
ESR-150	200	88.70%	61.50%	92.50%
ESR-200	200	80.80%	54.50%	90.20%
Full-seq	100	81.00%	58.20%	86.70%

Table 13: **Scaling results.** All use ESR-100, LoRA, 3,200 problems. Best across training steps. Baseline performance shown for reference.

Student → Teacher	Math avg@4	BFCL full_acc	HE pass@1	MBPP+ pass@1
M-1.5B baseline	50.95%	2.70%	—	—
M-1.5B → Q3-1.7B	65.85%	61.30%	42.07%	44.44%
M-1.5B → Q3-4B	68.95%	45.80%	43.29%	47.62%
M-1.5B → Q3-8B	67.85%	57.00%	41.46%	44.44%
Q3-1.7B baseline	69.20%	59.80%	39.63%	44.71%
Q3-1.7B → Q3-4B	69.20%	73.70%	44.51%	47.62%
Q3-1.7B → Q3-8B	69.15%	74.00%	40.24%	48.15%
Q3-4B baseline	77.95%	73.00%	73.17%	55.56%
Q3-4B → Q3-8B	77.05%	76.80%	71.34%	56.35%

456 **C.3 Coding Results**457 **C.4 Function Calling Results**458 **C.5 Scaling Experiment Results**459 **C.6 Full-Sequence Degradation Details**

460 Table 14 summarises every full-rollout cell (across pairs, tasks, and regimes) for which we have a
461 per-step trajectory on disk and observe degradation across training. For each cell we report the *peak*
462 score (best checkpoint across training) and the *lowest* score (worst checkpoint after the peak), together
463 with the step indices at which they occur. Bold Δ values indicate cells where Full Rollout drops
464 below the student baseline. The Qwen-math $n=16$ row (boxed-repetition collapse) is documented in
465 greater detail in Table 15 below.

466 **Per-step trajectories** for the seven cells in Table 14:

- 467 • **Qwen-1.5B→Q3-1.7B, LoRA, HumanEval (pass@1):** 40.2 (s50) → 31.7 → 32.3 → 32.9
468 → 27.4 → 28.0 → 26.8 → 26.8 (s400).

Table 14: **Full-rollout collapses across training (peak $\rightarrow$ lowest).** Each row is a single Full Rollout experiment; we report the highest and lowest checkpoint scores across all saved steps. “ Δ ” is lowest $-$ peak. Cells with † end the run *below the student baseline*. Trajectories are taken from the per-step eval logs on the training cluster (docs/main_results.md, docs/scaling_results.md). ESR on the same cells (not shown here) stays within ± 1 pp of its peak across training; see Tables 9 and 11.

Pair	Regime	Task	Metric	Peak (step)	Lowest (step)	Δ
Qwen-1.5B $\rightarrow$ Q3-1.7B	LoRA	MATH-500 ($n=16$)	avg@4	65.60 (s50)	43.80 (s200) †	−21.80
Qwen-1.5B $\rightarrow$ Q3-4B	LoRA	MATH-500	avg@4	67.45 (s50)	54.45 (s100)	−13.00
Qwen-1.5B $\rightarrow$ Q3-4B	LoRA	MATH-500	maj@4	73.20 (s50)	63.40 (s200)	−9.80
Qwen-1.5B $\rightarrow$ Q3-1.7B	LoRA	HumanEval	pass@1	40.2 (s50)	26.8 (s350)	−13.4
Qwen-1.5B $\rightarrow$ Q3-1.7B	LoRA	HumanEval+	pass@1	35.4 (s50)	25.0 (s250)	−10.4
Qwen-1.5B $\rightarrow$ Q3-1.7B	FullIFT	HumanEval	pass@1	36.6 (s150)	30.5 (s300)	−6.1
Qwen-1.5B $\rightarrow$ Q3-1.7B	FullIFT	HumanEval+	pass@1	31.7 (s150)	26.2 (s250)	−5.5
Gemma-2-2B $\rightarrow$ Gemma-3-4B	LoRA	HumanEval	pass@1	22.0 (s50)	11.6 (s150) †	−10.4

† Lowest score is below the student baseline (Qwen-Math-1.5B MATH-500: 50.95%; Gemma-2-2B HumanEval: 23.78%).

- **Qwen-1.5B $\rightarrow$ Q3-1.7B, LoRA, HumanEval+ (pass@1):** 35.4 (s50) $\rightarrow$ 27.4 $\rightarrow$ 29.9 $\rightarrow$ 29.9 $\rightarrow$ 25.0 $\rightarrow$ 26.2 $\rightarrow$ 25.0 $\rightarrow$ 25.0 (s400).
- **Qwen-1.5B $\rightarrow$ Q3-1.7B, FullIFT, HumanEval (pass@1):** 32.3 $\rightarrow$ 32.3 $\rightarrow$ 36.6 (s150) $\rightarrow$ 36.0 $\rightarrow$ 31.1 $\rightarrow$ 30.5 $\rightarrow$ 31.7 $\rightarrow$ 31.1 (s400).
- **Qwen-1.5B $\rightarrow$ Q3-1.7B, FullIFT, HumanEval+ (pass@1):** 27.4 $\rightarrow$ 28.7 $\rightarrow$ 31.7 (s150) $\rightarrow$ 30.5 $\rightarrow$ 26.2 $\rightarrow$ 26.2 $\rightarrow$ 26.8 $\rightarrow$ 26.2 (s400).
- **Gemma-2-2B $\rightarrow$ Gemma-3-4B, LoRA, HumanEval (pass@1):** 22.0 (s50) $\rightarrow$ 14.6 $\rightarrow$ 11.6 $\rightarrow$ 11.6 (s200). Falls below the 23.78 student baseline at s100 and stays there.
- **Qwen-1.5B $\rightarrow$ Q3-4B, LoRA, MATH-500 avg@4:** 67.45 (s50) $\rightarrow$ 54.45 $\rightarrow$ 58.85 $\rightarrow$ 55.05 (s200) (boxed-repetition pattern; ESR-100 on the same pair climbs to 68.95).
- **Qwen-1.5B $\rightarrow$ Q3-1.7B, LoRA, MATH-500 ($n=16$) avg@4:** 65.60 (s50) $\rightarrow$ 46.30 $\rightarrow$ 49.70 $\rightarrow$ 43.80 (s200). Boxed-repetition collapse; statistics in Table 15.

Table 15: **Boxed repetition progression** in full-sequence LoRA distillation on MATH-500 ($n=16$, bs=16). Companion to the corresponding row of Table 14.

Statistic	Step 50	Step 100	Step 150	Step 200
Multi-boxed responses	2.9%	91.5%	80.5%	89.1%
Avg \boxed{} per response	1.0	88.1	54.1	58.0
Repetitive responses	5.9%	78.8%	58.3%	77.5%
Avg response length (chars)	1,388	4,023	3,585	4,558
Official avg@4	65.60%	46.30%	49.70%	43.80%
Relaxed avg@4 (GT in text)	74.60%	70.90%	71.90%	70.30%

D KL-Profile Diagnostic for Choosing N

A practical question for a new student–teacher pair is: what value of N should I use? Our position-sweep (Section 4.3) shows a wide plateau on Qwen math ($N \in [50, 200]$, avg@4 within ~ 1 pp); the same plateau appears on Gemma math ($N \approx 100$). These plateau locations coincide with the position at which the cumulative per-position KL $\sum_{k \leq N} \mathbb{E}[\text{KL}_k]$ reaches $\sim 45\%$ of the full-trajectory total, measured on a small (~ 1 k trajectory) pre-training rollout. This suggests a cheap rule of thumb: run a 1k-trajectory per-position KL scan before training, and pick N at the 45% cumulative-KL point. We verify this on two cells (Qwen, Gemma math) and leave a full six-cell verification ($\{\text{Qwen, Gemma}\} \times \{\text{math, coding, funcall}\}$) as a short follow-up (**TBD: KL-profile scan**); the diagnostic is reported here as a practical recipe rather than a claim.

E Token Classification Methodology

We classify each token into six categories based on string matching:

1. **planning**: Reasoning keywords (“To”, “Let”, “First”, “Step”, “We”, “Given”, “Therefore”, “Thus”, “Since”).
2. **structural**: Punctuation, whitespace, formatting tokens.
3. **math_number**: Digits (0–9).
4. **math_operator**: Arithmetic operators (+, −, ×, /, =).
5. **math_latex**: LaTeX delimiters (\(, \[).
6. **continuation**: All others.

Table 16: Mean KL by token category and position range.

Category	0–4	5–19	20–49	50–99	100–199	200–499
planning	4.50	0.79	1.49	1.66	1.49	2.37
structural	3.26	1.46	1.60	0.93	0.60	0.86
math_number	1.49	0.60	0.74	0.28	0.17	0.13
math_operator	7.30	1.84	0.81	0.37	0.21	0.14
math_latex	8.84	9.23	6.50	4.95	2.97	1.87
continuation	1.94	1.12	1.19	0.89	0.70	0.48

Table 17: Top 20 highest-KL tokens (minimum 50 occurrences across 10,000 trajectories).

Rank	Token	Category	Count	Mean KL
1	“Solution”	planning	152	21.93
2	“Analysis”	continuation	125	16.49
3	\[	math_latex	7,152	13.21
4	“examines”	continuation	74	11.51
5	“He”	continuation	150	10.80
6	\(	math_latex	21,243	10.30
7	“First”	planning	1,706	9.98
8	“tests”	continuation	52	8.71
9	\\	math_latex	82	8.68
10	“There”	continuation	201	8.28
11	“Therefore”	planning	4,913	7.95
12	“Identify”	continuation	1,345	7.78
13	“To”	planning	8,806	6.90
14	“When”	continuation	89	6.62
15	“This”	continuation	621	6.34
16	“Thus”	planning	2,547	6.24
17	“First” (space)	planning	259	6.10
18	“To” (space)	planning	1,174	5.37
19	“The”	planning	2,626	5.25
20	“Next”	planning	1,753	5.03

NeurIPS Paper Checklist

The checklist is designed to encourage best practices for responsible machine learning research, addressing issues of reproducibility, transparency, research ethics, and societal impact. Do not remove the checklist: **The papers not including the checklist will be desk rejected.** The checklist should follow the references and follow the (optional) supplemental material. The checklist does NOT count towards the page limit.

Please read the checklist guidelines carefully for information on how to answer these questions. For each question in the checklist:

- You should answer [Yes], [No], or [N/A].
- [N/A] means either that the question is Not Applicable for that particular paper or the relevant information is Not Available.
- Please provide a short (1–2 sentence) justification right after your answer (even for [N/A]).

The checklist answers are an integral part of your paper submission. They are visible to the reviewers, area chairs, senior area chairs, and ethics reviewers. You will also be asked to include it (after eventual revisions) with the final version of your paper, and its final version will be published with the paper.

The reviewers of your paper will be asked to use the checklist as one of the factors in their evaluation. While [Yes] is generally preferable to [No], it is perfectly acceptable to answer [No] provided a proper justification is given (e.g., error bars are not reported because it would be too computationally expensive” or “we were unable to find the license for the dataset we used”). In general, answering [No] or [N/A] is not grounds for rejection. While the questions are phrased in a binary way, we acknowledge that the true answer is often more nuanced, so please just use your best judgment and write a justification to elaborate. All supporting evidence can appear either in the main paper or the supplemental material, provided in appendix. If you answer [Yes] to a question, in the justification please point to the section(s) where related material for the question can be found.

IMPORTANT, please:

- **Delete this instruction block, but keep the section heading “NeurIPS Paper Checklist”.**
- **Keep the checklist subsection headings, questions/answers and guidelines below.**
- **Do not modify the questions and only use the provided macros for your answers.**

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [TODO]

Justification: [TODO]

Guidelines:

- The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [TODO]

Justification: [TODO]

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [TODO]

Justification: [TODO]

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [TODO]

Justification: [TODO]

Guidelines:

- The answer [N/A] means that the paper does not include experiments.

- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
 - (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [TODO]

Justification: [TODO]

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).

- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer **[Yes]** if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.

- 704 • The paper should disclose whether the full research project required more compute
705 than the experiments reported in the paper (e.g., preliminary or failed experiments that
706 didn't make it into the paper).

707 **9. Code of ethics**

708 Question: Does the research conducted in the paper conform, in every respect, with the
709 NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines?>

710 Answer: **[TODO]**

711 Justification: **[TODO]**

712 Guidelines:

- 713 • The answer [N/A] means that the authors have not reviewed the NeurIPS Code of
714 Ethics.
715 • If the authors answer [No], they should explain the special circumstances that require a
716 deviation from the Code of Ethics.
717 • The authors should make sure to preserve anonymity (e.g., if there is a special consid-
718 eration due to laws or regulations in their jurisdiction).

719 **10. Broader impacts**

720 Question: Does the paper discuss both potential positive societal impacts and negative
721 societal impacts of the work performed?

722 Answer: **[TODO]**

723 Justification: **[TODO]**

724 Guidelines:

- 725 • The answer [N/A] means that there is no societal impact of the work performed.
726 • If the authors answer [N/A] or [No], they should explain why their work has no societal
727 impact or why the paper does not address societal impact.
728 • Examples of negative societal impacts include potential malicious or unintended uses
729 (e.g., disinformation, generating fake profiles, surveillance), fairness considerations
730 (e.g., deployment of technologies that could make decisions that unfairly impact specific
731 groups), privacy considerations, and security considerations.
732 • The conference expects that many papers will be foundational research and not tied
733 to particular applications, let alone deployments. However, if there is a direct path to
734 any negative applications, the authors should point it out. For example, it is legitimate
735 to point out that an improvement in the quality of generative models could be used to
736 generate Deepfakes for disinformation. On the other hand, it is not needed to point out
737 that a generic algorithm for optimizing neural networks could enable people to train
738 models that generate Deepfakes faster.
739 • The authors should consider possible harms that could arise when the technology is
740 being used as intended and functioning correctly, harms that could arise when the
741 technology is being used as intended but gives incorrect results, and harms following
742 from (intentional or unintentional) misuse of the technology.
743 • If there are negative societal impacts, the authors could also discuss possible mitigation
744 strategies (e.g., gated release of models, providing defenses in addition to attacks,
745 mechanisms for monitoring misuse, mechanisms to monitor how a system learns from
746 feedback over time, improving the efficiency and accessibility of ML).

747 **11. Safeguards**

748 Question: Does the paper describe safeguards that have been put in place for responsible
749 release of data or models that have a high risk for misuse (e.g., pre-trained language models,
750 image generators, or scraped datasets)?

751 Answer: **[TODO]**

752 Justification: **[TODO]**

753 Guidelines:

- 754 • The answer [N/A] means that the paper poses no such risks.

- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. Institutional review board (IRB) approvals or equivalent for research with human subjects

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [TODO]

Justification: [TODO]

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. Declaration of LLM usage

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [TODO]

Justification: [TODO]

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.